Kadane's Algorithm

Maximum Subarray	——>	Medium
Maximum Sum Circular Subarray	——>	Medium

⇒ 53. Maximum Subarray Sum [Kadane's Algorithm]

Q Given an integer array nums, find the subarray with the largest sum, and return its sum.

| 1 | 2 | 3 | 4 | 5 | → n

subarrays are:

1, 2, 3, 4, 5

12, 23, 34, 45

123, 234, 345

1234, 2345

12345

Total no. of subarrays $= \frac{n(n+1)}{2}$

$n=5 \Rightarrow \frac{5 \times 6}{2} = 15$ subarrays

- Subarrays can start from any index.

0	1	2	3	4
1	2	3	4	5
i				

subarrays are

1, 12, 13, 14, 15

$\boxed{T.C \Rightarrow O(n^3)}$

start index	end index
0	0, 1, 2, 3, 4
1	1, 2, 3, 4
2	2, 3, 4

	i			
1	2	3	4	5

subarrays are

2, 23, 24, 25

```
for(int start=0; start<n; start++){
  for(end=start; end<n; end++){
    ⇒ (start to end)
  }
}
```

DATE: __/__/__
PAGE: _____

to print all subarrays

```
int main(){
int n = 5;
int arr[5] = {1, 2, 3, 4, 5};
for(int st=0; st<n; st++){
    for(int end= st; end<n; end++){
        for(int i=st; i<=end ; i++){
            cout << arr[i];
        }
        cout << " ";
    }
    cout<<endl;
}
return 0;
}
```

⇒ | 3 | -4 | 5 | 4 | -1 | 7 | -8 |

↑ start ↑ end

add previous sum

sum ⇒ | 3 | -1 | 4 | 3 | 10 | 18 |

3-4 -1+5 4-1 3+7 10+8

T.C ⇒ $O(n^2)$

Code

```
int maxSum = INT_MIN;
for(int st=0 ; st<n ; st++) {
    int currSum=0;
    for(int end=st ; end<n ; end++){
        currSum += arr[end];
        maxSum = max( currSum , maxSum);
    }
}
cout << "max subarray sum= " << maxSum << endl;
return 0;
}
```

return

DATE: __/__/__
PAGE: _____

Kadane's Algorithm

optimised

(1) +ve + +ve → +ve ✓

(2) Small +ve + large -ve → -ve

↳ it always decrease the maximum sum so ~~reset~~ reset it to 0

(3) Small -ve + large +ve → +ve ✓

curr sum = 0 , max sum = $-\infty$

eg [3, -4, 5, 4, -1, 7, -8]

curr sum = 3
max sum = (3, $-\infty$) = 3

[3, -4, 5, 4, -1, 7, -8]	[3, -4, 5, 4, -1, 7, -8]	[3, -4, 5, 4, -1, 7, -8]
curr sum = 3 - 4 = -1	curr sum = ~~0~~ 5	curr sum = 5 + 4 = 9
max sum = (-1, 3) = 3	max sum = (5, 3) = 5	max sum = (9, 5) = 9
and curr sum < 0		
So, curr sum = 0		

[3, -4, 5, 4, -1, 7, -8]	[3, -4, 5, 4, -1, 7, -8]	[3, -4, 5, 4, -1, 7, -8]
curr sum = 9 - 1 = 8	curr sum = 8 + 7 = 15	curr sum = 15 + (-8) = 7
max sum = (8, 9) = 9	max sum = (15, 9) = 15	max sum = (7, 15) = 15

```
int maxSubArray(vector<int> & nums){
 int currSum = 0,    maxSum = INT_MIN;
 for(int val : nums){        (answer store)
        currSum += val;
        maxSum = max(currSum, maxSum);
```

DATE: __/__/__

PAGE:

```
        if (currsum < 0) {
            currsum = 0;
        }
    }
    return maxsum;
}
```

T.C ⇒ O(n)

↳ linear